

Worked Solutions

Data Structures

Set A: Ensemble | CSE Semester 3

Companion to `ds-ensemble.pdf`. Question numbers match that paper. Equivalent correct methods are acceptable. These are independent worked solutions, not an official university marking scheme.

Questions 1–5: MCQ answer key, 1 mark each (5 marks total). Questions 6 onward: theory solutions (25 marks total).

1. Undo data structure

Correct option: (B). A stack removes the most recently pushed action first (LIFO).

2. First undo

Correct option: (D). C is the most recent edit and is at the top of the stack.

3. Redo stack order

Correct option: (A). The first undo pushes C onto redo; the second pushes B above C.

4. Redo operation

Correct option: (C). Redo pops the top action, B, and puts it back on undo.

5. Edit after undo

Correct option: (B). A new edit starts a new editing branch, so the previously undone future actions are discarded.

6. Nested employee structure

The outer structure stores employee details and the nested `Date` stores joining date. This program assumes a valid calendar date and a one-word name of at most 39 characters. The input-conversion count is checked.

```
#include <stdio.h>
typedef struct { int day, month, year; } Date;
typedef struct {
    int id;
    char name[40];
    Date joined;
} Employee;
int main(void) {
    Employee e;
    /* Input: id, one-word name, day, month, year. */
    if (scanf("%d %39s %d %d %d", &e.id, e.name,
            &e.joined.day, &e.joined.month,
            &e.joined.year) != 5) return 1;
    printf("%d %s %02d/%02d/%04d\n", e.id, e.name,
            e.joined.day, e.joined.month, e.joined.year);
    return 0;
}
```

7. Dynamic matrix and pointer arithmetic

A pointer to an array of three doubles provides contiguous dynamic storage for the three subjects. The expression $*(*(marks + i) + j)$ accesses one mark using pointer arithmetic. Time is $O(n)$ for three subjects and storage is $O(n)$.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
int main(void) {
    int n;
    if (scanf("%d", &n) != 1 || n <= 0) return 1;
    if ((size_t)n > SIZE_MAX / (3 * sizeof(double))) return 1;
    double (*marks)[3] = malloc((size_t)n * sizeof *marks);
    if (!marks) return 1;
    for (int i = 0; i < n; ++i) {
        double sum = 0;
        for (int j = 0; j < 3; ++j) {
            if (scanf("%lf", *(marks + i) + j) != 1) {
                free(marks); return 1;
            }
            sum += (*(marks + i) + j);
        }
        printf("Student %d average: %.2f\n", i + 1, sum / 3);
    }
    free(marks);
    return 0;
}
```

8. Dynamic string array and concatenation

Allocate two fixed-capacity rows dynamically, then allocate the exact combined length plus one byte for the null terminator. The pointer `dst` advances through the result as each character is copied. Inputs must each fit in 99 characters before their newline.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main(void) {
    /* Two rows of up to 99 characters, including spaces. */
    char (*s)[100] = malloc(2 * sizeof *s);
    if (!s) return 1;
    if (!fgets(*s, 100, stdin) || !fgets(*(s + 1), 100, stdin)) {
        free(s); return 1;
    }
    *(*s + strcspn(*s, "\n")) = '\0';
    *(*s + 1) + strcspn(*(s + 1), "\n")) = '\0';
    char *result = malloc(strlen(*s) + strlen(*(s + 1)) + 1);
    if (!result) { free(s); return 1; }
    char *dst = result;
    for (int row = 0; row < 2; ++row)
        for (const char *src = *(s + row); *src; ++src)
            *dst++ = *src;
    *dst = '\0';
    puts(result);
    free(result); free(s);
    return 0;
}
```

9. Playlist insertion and deletion

Append preserves insertion order. A pointer to the current link permits deletion of the head and consecutive short songs without a separate head special case. Deletion and display take $O(n)$ time; this simple append scans to the tail and takes $O(n)$ per insertion. Enter 180 in the demonstration to retain only Evening.

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Song {
    int id, duration;
    char title[80];
    struct Song *next;
} Song;

int append(Song **head, int id, const char *title, int duration) {
    Song *node = malloc(sizeof *node);
    if (!node) return 0;
    node->id = id; node->duration = duration; node->next = NULL;
    snprintf(node->title, sizeof node->title, "%s", title);
    Song **link = head;
    while (*link) link = &(*link)->next;
    *link = node;
    return 1;
}

void removeShort(Song **head, int limit) {
    Song **link = head;
    while (*link) {
        if ((*link)->duration < limit) {
            Song *old = *link;
            *link = old->next;
            free(old);
        } else link = &(*link)->next;
    }
}

void display(const Song *p) {
    for (; p; p = p->next)
        printf("%d %s %d\n", p->id, p->title, p->duration);
}

void clear(Song **head) {
    while (*head) {
        Song *old = *head; *head = old->next; free(old);
    }
}

int main(void) {
    Song *head = NULL;
    if (!append(&head, 1, "Dawn", 120) ||
        !append(&head, 2, "Evening", 240) ||
        !append(&head, 3, "Night", 150)) {
        clear(&head); return 1;
    }
    int limit;
    if (scanf("%d", &limit) != 1) { clear(&head); return 1; }
    removeShort(&head, limit);
    display(head);
    clear(&head);
    return 0;
}
```

10. Recent contacts and traversal

Insert at the head in $O(1)$ time. Forward and reverse printing take $O(n)$ time. The recursive reverse printer uses $O(n)$ call-stack space and does not change the list. Each call adds a contact record; moving an already present contact would additionally require locating and unlinking its previous node.

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Contact {
    int id;
    char name[40];
    struct Contact *next;
} Contact;
int addRecent(Contact **head, int id, const char *name) {
    Contact *p = malloc(sizeof *p);
    if (!p) return 0;
    p->id = id;
    snprintf(p->name, sizeof p->name, "%s", name);
    p->next = *head; *head = p;
    return 1;
}
void forward(const Contact *p) {
    for (; p; p = p->next) printf("%d %s\n", p->id, p->name);
}
void reversePrint(const Contact *p) {
    if (!p) return;
    reversePrint(p->next);
    printf("%d %s\n", p->id, p->name);
}
```

11. Infix to postfix with precedence functions

Pop while in-stack precedence is at least incoming precedence. Giving incoming exponentiation precedence 6 and in-stack exponentiation precedence 5 keeps equal exponentiation operators on the stack, producing right associativity. Equal-precedence addition, subtraction, multiplication and division pop, producing left associativity. This C99 implementation handles valid expressions with single-character operands and optional parentheses.

```
#include <ctype.h>
#include <stddef.h>
#include <string.h>
int InStackPrecedence(char op) {
    switch (op) {
        case '+': case '-': return 1;
        case '*': case '/': return 3;
        case '^': return 5;
        case '(': return 0;
        default: return -1;
    }
}
int IncomingPrecedence(char op) {
    switch (op) {
        case '+': case '-': return 1;
        case '*': case '/': return 3;
        case '^': return 6; /* Do not pop an existing '^'. */
        case '(': return 7;
        default: return -1;
    }
}
/* Valid expressions, single-character operands; output has
   space for strlen(infix)+1 bytes. Parentheses are supported. */
void Infix_Postfix(char infix[], char postfix[]) {
    char stack[strlen(infix) + 1];
    size_t top = 0, out = 0;
    for (size_t i = 0; infix[i]; ++i) {
        char c = infix[i];
        if (isspace((unsigned char)c)) continue;
        if (isalnum((unsigned char)c)) postfix[out++] = c;
        else if (c == '(') stack[top++] = c;
        else if (c == ')') {
            while (top && stack[top - 1] != '(')
                postfix[out++] = stack[--top];
            if (top) --top;
        } else {
            while (top && stack[top - 1] != '(' &&
                InStackPrecedence(stack[top - 1]) >=
                IncomingPrecedence(c))
                postfix[out++] = stack[--top];
            stack[top++] = c;
        }
    }
    while (top) postfix[out++] = stack[--top];
    postfix[out] = '\0';
}
```

For A^B^C , the output is $ABC^^$. For $A-B-C$, it is $AB-C-$. Each token is pushed and popped at most once, so time and auxiliary space are $O(n)$.

12. Duplicate parentheses

Here duplicate means a redundant pair enclosing no operator at its own nesting level, including an extra wrapper around a parenthesized expression. The input is assumed syntactically valid and balanced. Pop a closed group, check whether an operator occurred, and reduce a useful group to one operand marker. Time and stack space are $O(n)$.

```
#include <stdbool.h>
#include <ctype.h>
#include <string.h>
/* Input is a syntactically valid, balanced expression.
   Redundant pair means a pair enclosing no operator at its
   own nesting level: (a), ((a+b)), and () are detected. */
bool isDuplicate(char s[]) {
    char stack[strlen(s) + 1];
    size_t top = 0;
    for (size_t i = 0; s[i]; ++i) {
        char c = s[i];
        if (isspace((unsigned char)c)) continue;
        if (c != ')') stack[top++] = c;
        else {
            bool hasOperator = false;
            while (top && stack[top - 1] != '(') {
                char v = stack[--top];
                if (strchr("+-*/^", v)) hasOperator = true;
            }
            if (top) --top;
            if (!hasOperator) return true;
            stack[top++] = 'x'; /* One reduced subexpression. */
        }
    }
    return false;
}
```

Examples: ((a+b)) returns true; (a+b) returns false; (a) returns true under the stated convention.

13. Activation records and LIFO

Calls push frames in this order: main, fun2, fun1. Returns remove frames in the reverse order: fun1, fun2, main. The output is:

```
Inside fun1
Inside fun2
Inside main
```

A frame holds the return address, parameters, local variables and saved execution state as required by the calling convention. A queue would remove the oldest caller first, although that caller is suspended awaiting its callee, so it does not represent normal nested call/return order.